BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM

KHOA ĐIỆN – ĐIỆN TỬ



BÁO CÁO CUỐI KỲ MÔN HỌC

HỆ THỐNG NHÚNG

STANDBY MODE - LOW POWER

MÃ MÔN HỌC: EMSY435664

SÁNG THỨ 6, ĐỢT 2, HỌC KÌ 1, NĂM HỌC 2025-2026

Thực hiện:

Nguyễn Ngọc Trường Phi	**23119187**
Hà Quang Huy	**23119147**
Võ Trung Nguyên	**23119180**
Vũ Minh Quang	**23119198**

Giảng viên hướng dẫn: ThS. Huỳnh Hoàng Hà

Thành phố Hồ Chí Minh, Tháng 1 năm 2026

MỤC LỤC

MỤC LỤC ..... **i**
DANH MỤC HÌNH ẢNH ..... **ii**
CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG CUNG CẤP NĂNG LƯỢNG. **1**
1.1. Các miền nguồn điện áp. 1
1.2. Bộ điều áp nguồn (Voltage Regulator) và các trạng thái hoạt động 1
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT **3**
2.1. Tổng quan về chế độ Standby Mode. 3
2.1.1. Khái niệm. *3*
2.1.2. Đặc điểm. *3*
2.1.3. Nguyên lý hoạt động. *3*
2.1.4. Bảng so sánh sự khác biệt giữa các chế độ trong Low-Power *4*
2.2. Kiến trúc điều khiển năng lượng tại lõi Cortex-M3. 5
2.3. Kĩ thuật giảm tiêu thụ điện năng của chế độ Standby. 6
2.4. Cơ chế thực thi lệnh vào chế độ Standby. 6
2.5. Các nguồn đánh thức chế độ Standby. 6
2.6 Cơ chế thực thi lệnh đánh thức hệ thống. 7
CHƯƠNG 3: THIẾT KẾ CHƯƠNG TRÌNH THỰC NGHIỆM CHẾ ĐỘ STANBY MODE. **8**
3.1. Sơ đồ thiết kế phần cứng. 8
3.2. Flow chart. 9
3.3. Cấu hình CubeMx. 12
3.4. Phân tích chương trình. 16
CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ PHÂN TÍCH. **27**
4.1. Kết quả phần cứng 27
4.2. Kết quả đo lường tiêu thụ dòng điện của chế độ Standby. 27
4.3. Phân tích kết quả đo lường dòng điện tiêu thụ. 28
CHƯƠNG 5: KẾT LUẬN VÀ ĐÁNH GIÁ. **29**
5.1. Kết luận giá trị dòng điện tiêu thụ. 29
5.2. Đánh giá hiệu quả của chế độ Standby. 29
5.3. Ứng dụng thực tế của chế độ Standby. 29
TÀI LIỆU THAM KHẢO **30**

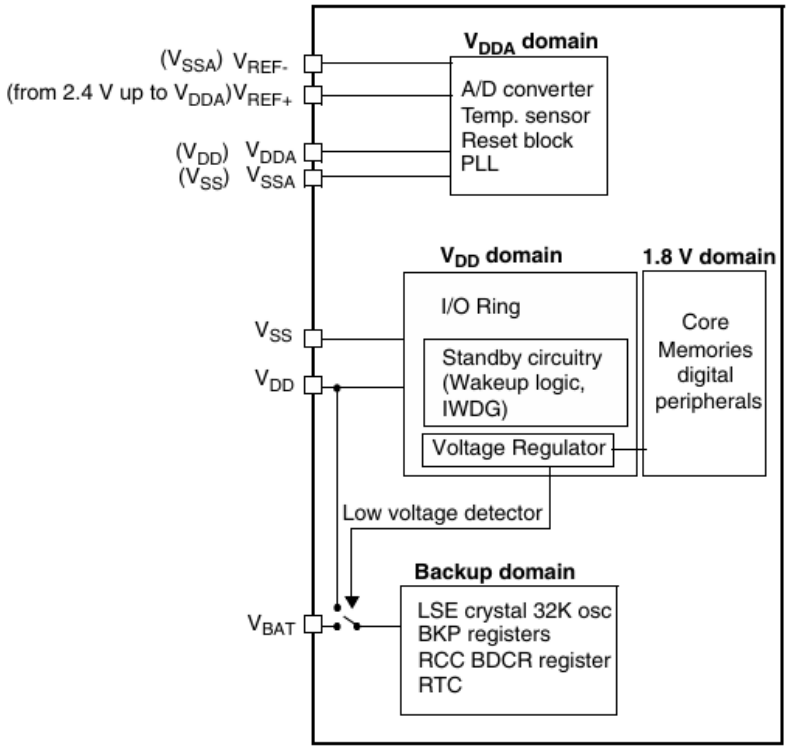
DANH MỤC HÌNH ẢNH.

Hình 1. Tổng quan về nguồn điện.....1
Hình 2. Kiến trúc điều khiển năng lượng lõi Cortex-M3.....5
Hình 3. Sơ đồ thiết kế phần cứng.....8
Hình 4: Flowchart 9
Hình 5: Flowchart 11
Hình 6: Flowchart 12
Hình 7: Cấu hình GPIO.....13
Hình 8: Cấu hình NVIC 13
Hình 9: Cấu hình RCC.....14
Hình 10: Cấu hình SYS.....14
Hình 11: Cấu hình Timer1 15
Hình 12: Cấu hình Clock 15
Hình 13: Kết quả phần cứng 27
Hình 14: dòng tiêu thụ Run mode.....27
Hình 15: Dòng tiêu thụ Standby mode 28

CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG CUNG CẤP NĂNG LƯỢNG.

1.1. Các miền nguồn điện áp.

-Hệ thống nguồn của STM32F103C8T6 được thiết kế chia tầng để tối ưu hóa công suất tiêu thụ:



Hình 1. Tổng quan về nguồn điện.

-Miền nguồn chính (V_{DD}): Có dải điện áp từ 2.0V đến 3.6V, cấp nguồn trực tiếp cho các chân I/O và bộ điều áp nguồn nội bộ.

-Miền nguồn Analog (V_{DDA}): Có giá trị từ 2.4V đến 3.6V. Để cấp cho bộ ADC (tối thiểu 2.4V), các bộ dao động RC và PLL.

-Miền 1.8V: Được cấp bởi bộ điều áp, cung cấp năng lượng cho lõi Cortex-M3, bộ nhớ và các ngoại vi kỹ thuật số.

-Miền dự phòng (V_{BAT}): Có dải từ 1.8V đến 3.6V, dùng để duy trì bộ RTC và các thanh ghi dự phòng (BKP) khi nguồn V_{DD} chính bị ngắt.

1.2. Bộ điều áp nguồn (Voltage Regulator) và các trạng thái hoạt động

Bộ điều chỉnh điện áp luôn được kích hoạt sau khi reset. Trong STM32F103C8T6 nó đóng vai trò quan trọng trong việc kiểm soát năng lượng tại miền 1.8V thông qua ba chế độ hoạt động sau:

-Trong Run mode: Bộ điều áp cung cấp đầy đủ năng lượng để các ngoại vi hoạt động ở công suất cao nhất.

-Trong Sleep mode: Bộ điều áp vẫn được duy trì ở trạng thái hoạt động bình thường và tiếp tục cấp điện cho miền 1.8 V. Lõi CPU tạm dừng thực thi lệnh, trong khi các ngoại vi và xung nhịp hệ thống vẫn có thể tiếp tục hoạt động.

-Trong Stop mode: Bộ điều chỉnh cung cấp điện năng thấp cho miền 1,8 V, giữ nguyên nội dung của các thanh ghi và SRAM.

-Trong Standby mode: Bộ điều áp bị ngắt hoàn toàn. Đầu ra của nó chuyển sang trạng thái trở kháng cao, làm cho miền 1.8V không còn điện năng và dẫn đến mức tiêu thụ dòng điện cực thấp (chỉ khoảng 2 µA ở 3.3V).

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về chế độ Standby Mode.

2.1.1. Khái niệm.

-Standby mode là chế độ tiêu thụ điện năng thấp nhất trong các low-power modes của STM32.

-Ở Standby mode sẽ giảm dòng tiêu thụ xuống mức cực thấp (thường chỉ vài µA).

- Khi vào Standby mode:

- CPU dừng hoàn toàn
- Hầu hết các ngoại vi bị tắt
- RAM bị mất dữ liệu

2.1.2. Đặc điểm.

-Tiêu thụ điện năng cực thấp.

-RAM không được giữ, mất dữ liệu.

-Thanh ghi CPU, ngoại vi bị tắt hoàn toàn. Khi wake-up thì tương đương với reset hệ thống.

-Không duy trì xung clock.

-Chỉ Backup registers và RTC (nếu bật) còn giữ lại.

2.1.3. Nguyên lý hoạt động.

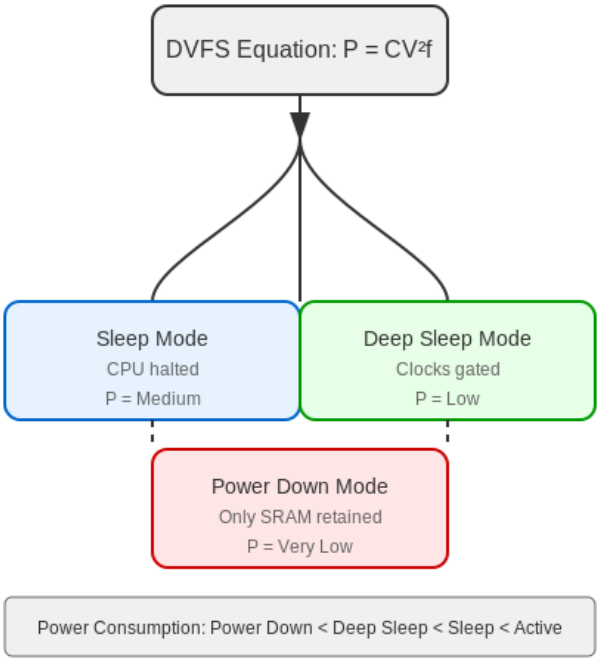
-Khi Standby Mode được kích hoạt, vi điều khiển ngắt cấp nguồn cho vùng lõi (VCORE) bằng cách tắt bộ điều áp nội. Do đó, CPU, SRAM và các ngoại vi phụ thuộc vào clock hệ thống đều bị tắt hoàn toàn. Chỉ các khối tối thiểu như mạch giám sát nguồn (BOR), RTC (nếu được cấp nguồn VBAT) và mạch phát hiện tín hiệu đánh thức vẫn còn hoạt động nhằm duy trì khả năng giám sát và đánh thức hệ thống.

-Khi xảy ra một sự kiện đánh thức hợp lệ như cạnh lên tại chân wake-up (PA0), báo thức từ RTC hoặc tín hiệu reset, khi đó vi điều khiển thoát khỏi Standby Mode và khởi động lại hoàn toàn từ trạng thái reset. Toàn bộ chương trình được thực thi lại từ đầu, tương tự như một lần reset cứng của hệ thống.

2.1.4. Bảng so sánh sự khác biệt giữa các chế độ trong Low-Power.

Đặc tính	**Chế độ Sleep**	**Chế độ Stop**	**Chế độ Standby**
Cách thức đi vào (Entry)	Lệnh WFI hoặc WFE.	Bit SLEEPDEEP + bit PDDS=0 + (WFI hoặc WFE).	Bit SLEEPDEEP + bit PDDS=1 + (WFI hoặc WFE).
Cơ chế đánh thức (Wakeup)	Bất kỳ ngắt (Interrupt) hoặc sự kiện (Event) nào.	Bất kỳ đường EXTI nào (cấu hình trong thanh ghi EXTI), báo thức RTC, USB wakeup.	Chân WKUP (cạnh lên), báo thức RTC, Reset ngoài (NRST), IWDG Reset.
Ảnh hưởng đến xung nhịp 1.8V	Chỉ dừng xung nhịp CPU; các ngoại vi khác vẫn hoạt động.	Dừng tất cả xung nhịp trong miền 1.8V (tắt PLL, HSI và HSE).	Ngắt nguồn hoàn toàn miền 1.8V.
Ảnh hưởng đến xung nhịp VDD	Không ảnh hưởng.	Tắt các bộ dao động HSI và HSE.	Tắt các bộ dao động HSI, HSE và PLL.
Bộ điều áp (Regulator)	Bật (ON).	Bật hoặc ở chế độ năng lượng thấp.	Tắt hoàn toàn (OFF).
Dữ liệu (Context)	Được bảo toàn hoàn toàn.	Bảo toàn nội dung SRAM và các thanh ghi.	Bị mất, ngoại trừ miền Backup và Standby backup.
Dòng tiêu thụ (*IDD*) điển hình tại 3.3V	~vài mA (72MHz, tắt hết ngoại vi).	~14 µA (Regulator ở chế độ Low-power).	~2 µA (Tắt LSI và RTC).

2.2. Kiến trúc điều khiển năng lượng tại lõi Cortex-M3.



Hình 2. Kiến trúc điều khiển năng lượng lõi Cortex-M3.

- Hai chế độ tiết kiệm năng lượng sâu nhất của STM32F103C8T6 là Stop và Standby thực chất được xây dựng dựa trên tính năng Deep Sleep (ngủ sâu) của kiến trúc Cortex-M3.

-Chế độ Stop (Deep Sleep + Clock Gating): Chế độ này dựa trên trạng thái Deep Sleep của lõi kết hợp với kỹ thuật ngắt toàn bộ xung nhịp ngoại vi. Ở trạng thái này, toàn bộ xung nhịp trong miền 1.8V bị dừng, nhưng nội dung SRAM và các thanh ghi vẫn được bảo toàn nhờ bộ điều áp hoạt động ở chế độ công suất thấp.

-Chế độ Standby (Deep Sleep + Power Down): Đây là mức ngủ sâu nhất, cũng dựa trên tính năng Deep Sleep của lõi nhưng đi kèm với việc vô hiệu hóa hoàn toàn bộ điều áp nội bộ. Toàn bộ miền 1.8V bị ngắt nguồn, khiến dữ liệu trong SRAM và các thanh ghi thông thường bị mất hoàn toàn, chỉ còn miền dự phòng (Backup domain) và mạch đánh thức Standby là tiếp tục hoạt động.

-Chế độ Sleep (CPU Sleep): Chế độ này được xây dựng trực tiếp từ trạng thái Sleep của lõi Cortex-M3, không sử dụng tính năng Deep Sleep. Khi vào chế độ này, chỉ có lõi CPU ngừng thực thi lệnh, trong khi xung nhịp hệ thống và xung nhịp ngoại vi vẫn tiếp tục hoạt động bình thường. Bộ điều áp nội bộ vẫn duy trì ở chế độ công suất đầy đủ, do đó toàn bộ nội dung trong SRAM và các thanh ghi của vi điều khiển được bảo toàn hoàn toàn.

2.3. Kĩ thuật giảm tiêu thụ điện năng của chế độ Standby.

-Chế độ Standby được thiết kế để đạt được mức tiêu thụ điện năng thấp nhất bằng cách áp dụng các kĩ thuật cắt giảm nguồn lực triệt để.

- Cơ chế cốt lõi là vô hiệu hóa hoàn toàn bộ điều áp (voltage regulator), dẫn đến việc ngắt nguồn miền kỹ thuật số 1.8V bao gồm lõi Cortex-M3, bộ nhớ và các ngoại vi số.

- Các bộ dao động tốc độ cao như PLL, HSI và HSE cũng bị tắt hoàn toàn.

- Tất cả các chân I/O ở chế độ này đều ở trạng thái trở kháng cao (high impedance), ngoại trừ chân Reset, chân đánh thức WKUP (nếu được bật) và các chân liên quan đến tính năng Tamper/hiệu chuẩn.

- Dòng tiêu thụ điển hình có thể giảm xuống chỉ còn khoảng 2 µA ở mức điện áp 3.3V khi tắt bộ dao động nội LSI và báo thức RTC.

2.4. Cơ chế thực thi lệnh vào chế độ Standby.

-Bước 1: Xóa cờ Wakeup cũ

```
__HAL_PWR_CLEAR_FLAG(PWR_FLAG_WU);
```

- Tránh hiện tượng vừa vào Standby Mode đã bị đánh thức ngay

-Bước 2: Cấu hình nguồn đánh thức.

```
HAL_PWR_EnableWakeUpPin(PWR_WAKEUP_PIN1);
```

- Cho phép chân WKUP làm nguồn đánh thức
- Vi điều khiển sẽ thoát chế độ Standby khi có cạnh lên tại chân này.

-Bước 3: Vào Standby Mode.

```
HAL_PWR_EnterSTANDBYMode();
```

- Vào chế độ Standby Mode, chỉ khởi động lại khi có các sự kiện đánh thức của Standby Mode.

2.5. Các nguồn đánh thức chế độ Standby.

Hệ thống chỉ có thể thoát khỏi trạng thái Standby khi xuất hiện một trong ba sự kiện đánh thức cụ thể sau:

- Cạnh lên trên chân WKUP (PA0): Một tín hiệu mức cao tại chân này sẽ kích hoạt quá trình đánh thức.

- Tín hiệu Reset ngoài trên chân NRST: Kéo chân Reset xuống mức thấp để khởi động lại hệ thống.

- Reset từ Watchdog độc lập (IWDG): Xảy ra khi bộ đếm IWDG kết thúc, buộc chip phải thức dậy và khởi động lại.

2.6 Cơ chế thực thi lệnh đánh thức hệ thống.

Khác với chế độ Sleep hay Stop, khi có đánh thức từ Standby, chương trình không tiếp tục thực hiện tại lệnh kế tiếp. Thay vào đó, chương trình sẽ thực hiện lại từ đầu giống như sau một chu kỳ Reset hệ thống.

-Bước 1: Xảy ra sự kiện đánh thức.

- Cạnh lên tại chân WKUP được cấu hình
- Reset ngoài (NRST)
- Reset từ WatchDog (IWDG)

-Bước 2: Vi điều khiển thoát chế độ Standby và Reset hệ thống.

- Bộ điều áp nội (Voltage Regulator) được bật trở lại
- Vùng lõi (VCORE) được cấp nguồn.
- Vi điều khiển STM32 khởi động lại hoàn toàn như reset cứng.

-Bước 3: Kiểm tra trạng thái thoát Standby trong chương trình.

```
__HAL_PWR_GET_FLAG(PWR_FLAG_SB);
```

- Cờ PWR_FLAG_SB cho biết hệ thống vừa thoát khỏi Standby Mode.
- Phân biệt Reset do Standby với Reset thông thường.

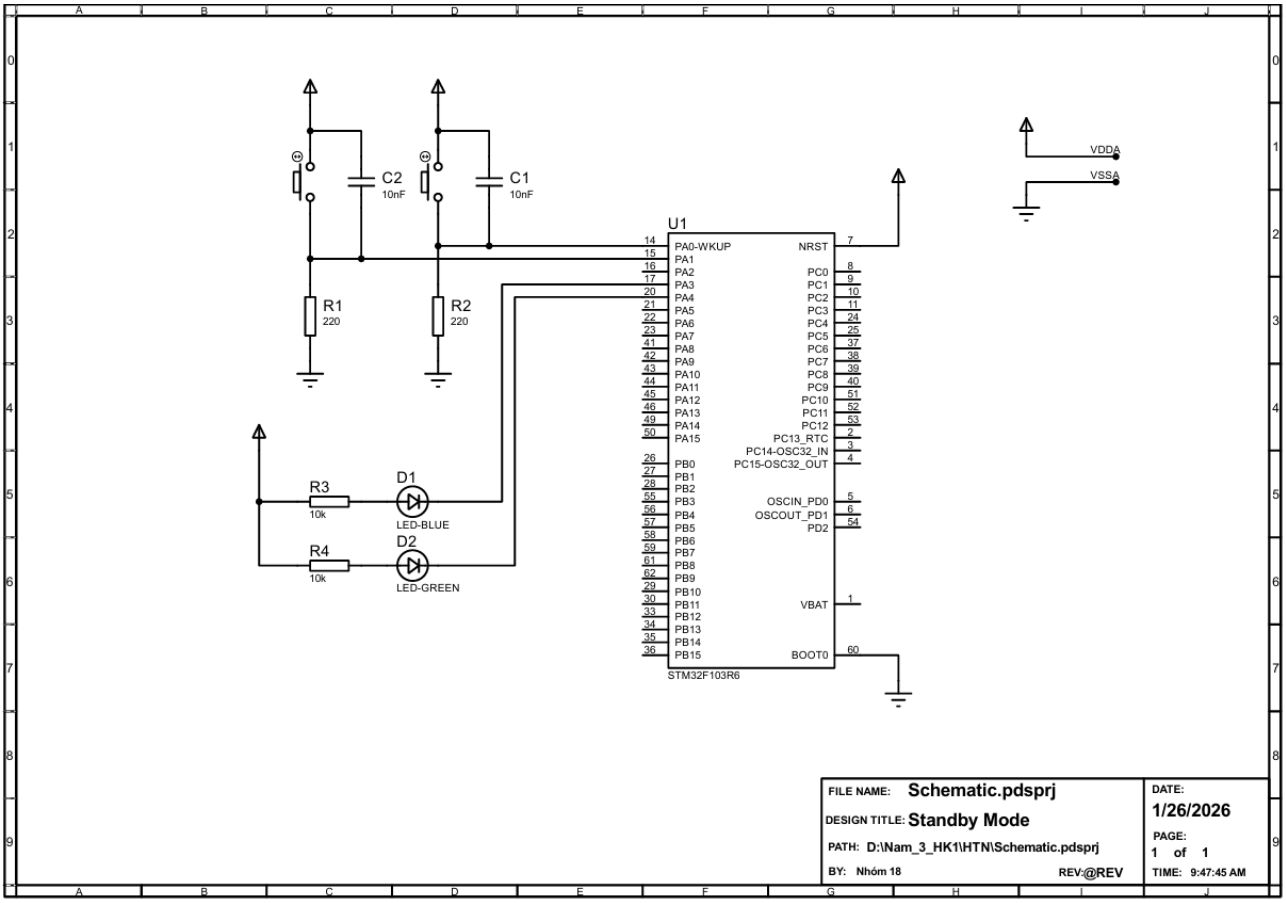
-Bước 4: Xóa cờ Standby.

```
__HAL_PWR_CLEAR_FLAG(PWR_FLAG_SB);
```

- Xóa cờ Standby sau khi đã kiểm tra trạng thái.
- Chuẩn bị cho lần vào Standby tiếp theo

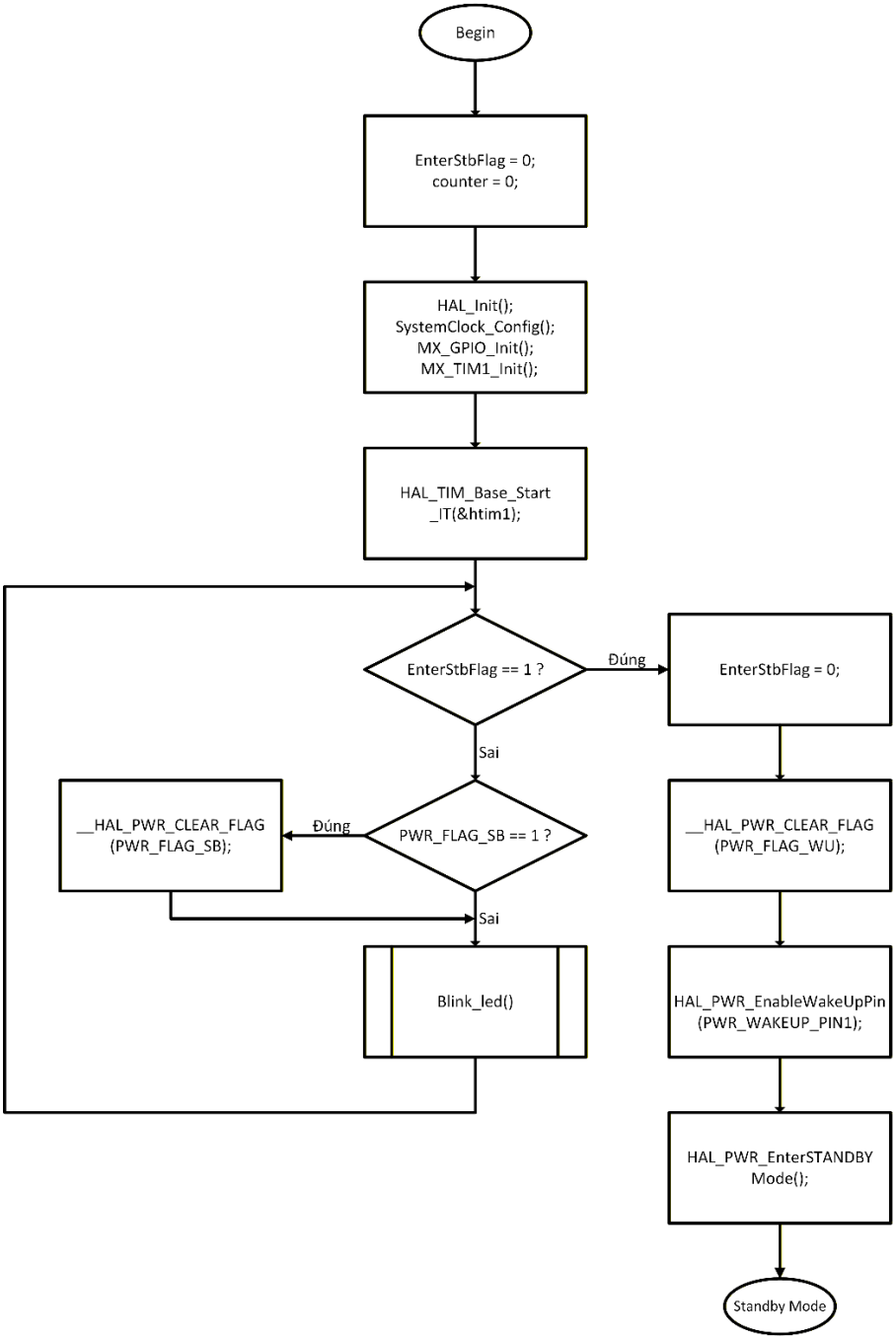
CHƯƠNG 3: THIẾT KẾ CHƯƠNG TRÌNH THỰC NGHIỆM CHẾ ĐỘ STANBY MODE.

3.1. Sơ đồ thiết kế phần cứng.



Hình 3. Sơ đồ thiết kế phần cứng.

3.2. Flow chart.



Hình 4: Flowchart

-Phân tích flowchart hàm main():

1. Khởi tạo hệ thống

-Chương trình bắt đầu bằng việc khởi tạo các biến điều khiển:

- EnterStbFlag được đặt bằng 0, dùng làm cờ báo yêu cầu vào chế độ Standby.
- counter được đặt bằng 0, dùng làm biến đếm để nhấp nháy led PA4.

-Sau đó, hệ thống tiến hành khởi tạo các thành phần cơ bản bao gồm:

- HAL_Init(): Khởi tạo thư viện HAL và cấu hình SysTick.
- SystemClock_Config(): Cấu hình xung nhịp hệ thống.
- MX_GPIO_Init() và MX_TIM1_Init(): Khởi tạo các ngoại vi GPIO và bộ định thời TIM1.
- HAL_TIM_Base_Start_IT(&htim1): Khởi tạo bộ định thời ngắt

2. Kiểm tra yêu cầu vào chế độ Standby.

-Kiểm tra điều kiện vào chế độ Standby:

-Nếu EnterStbFlag == 1:

- Đặt biến EnterStbFlag = 0: Tránh việc Standby lặp lại.
- Xóa cờ WU cũ
- Thiết lập chân đánh thức là PA0.
- Vào chế độ Standby và chỉ thoát chế độ Standby khi PA0 được nhấn.

-Nếu EnterStbFlag == 0:

- Tiếp tục lệnh tiếp theo của vòng lặp.

3. Kiểm tra trạng thái đánh thức.

-Kiểm tra cờ Standby trước đó:

-Nếu cờ PWR_FLAG_SB == 1:

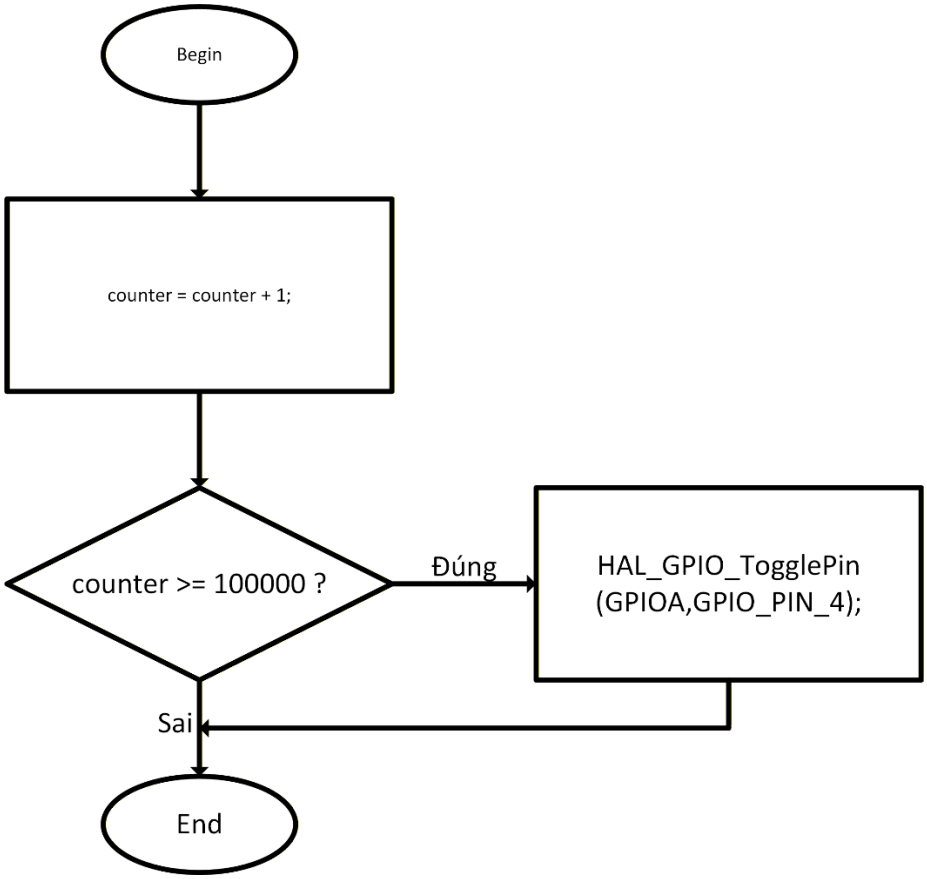
- Xóa cờ PWR_FLAG_SB về lại 0.

-Nếu cờ PWR_FLAG_SB == 1:

- Tiếp tục lệnh tiếp theo của vòng lặp.

4. Gọi hàm con.

-Gọi hàm con Blink_led(). Sau đó quay trở lại kiểm tra yêu cầu vào chế độ Standby tạo thành 1 vòng lặp chính.



Hình 5: Flowchart

-Phân tích Flowchart hàm con Blink_led():

1. Tăng biến đếm sau mỗi lần gọi hàm con.

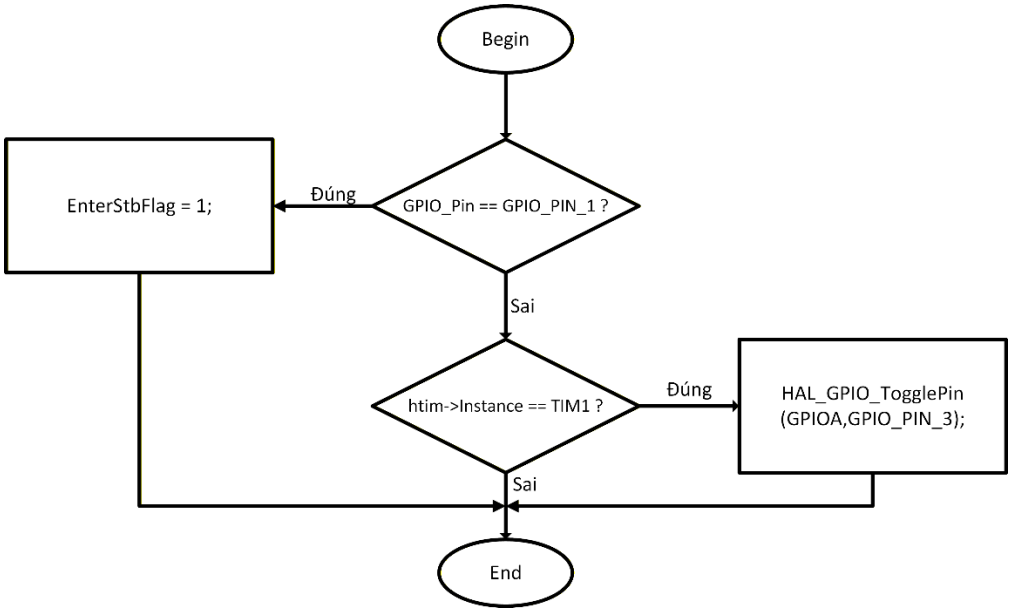
2. Kiểm tra biến đếm Counter >= 1000000 hay không.

-Nếu counter >= 1000000:

- Đảo trạng thái led chân PA4

-Nếu counter < 1000000:

- Kết thúc hàm con.



Hình 6: Flowchart

-Phân tích Flowchart hàm ngắt ngoài EXTI:

1. Kiểm tra sự kiện nút nhấn PA1:

-Nếu PA1 được nhấn:

- Biến EnterStbFlag được đặt = 1. Yêu cầu vào chế độ Standby.

-Nếu PA1 không được nhấn:

- Tiếp tục lệnh tiếp theo

2. Kiểm tra điều kiện tràn TIM1

-Nếu TIM1 tràn (500ms):

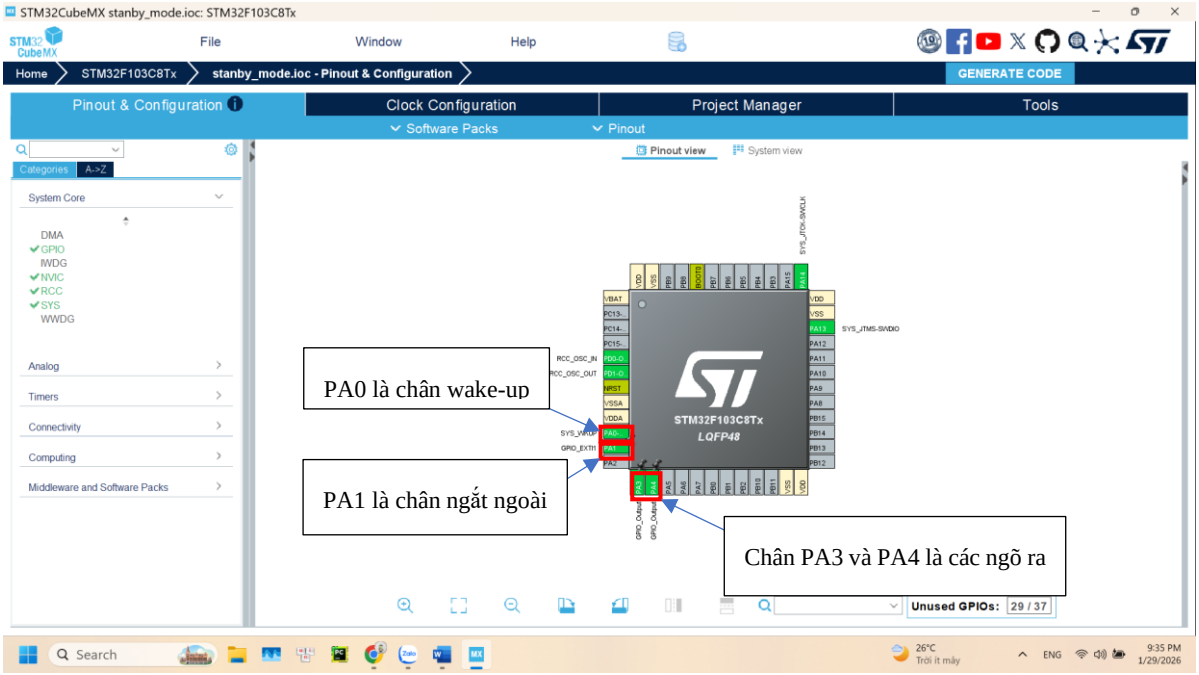
- Đảo trạng thái Led PA3.

-Nếu TIM1 chưa tràn:

- Kết thúc hàm ngắt ngoài.

3.3. Cấu hình CubeMx.

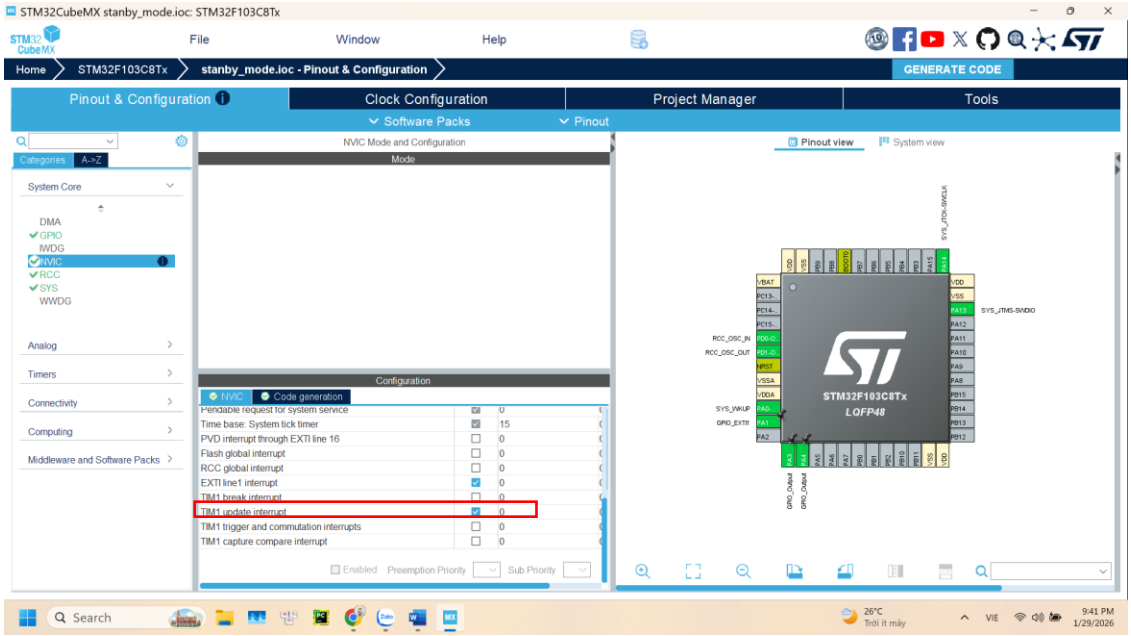
Bước 1: Cấu hình GPIO.



Hình 7: Cấu hình GPIO

Bước 2: Cấu hình NVIC

-NVIC bản chất là bộ điều khiển ngắt lồng nhau của lõi Cortex-M3.

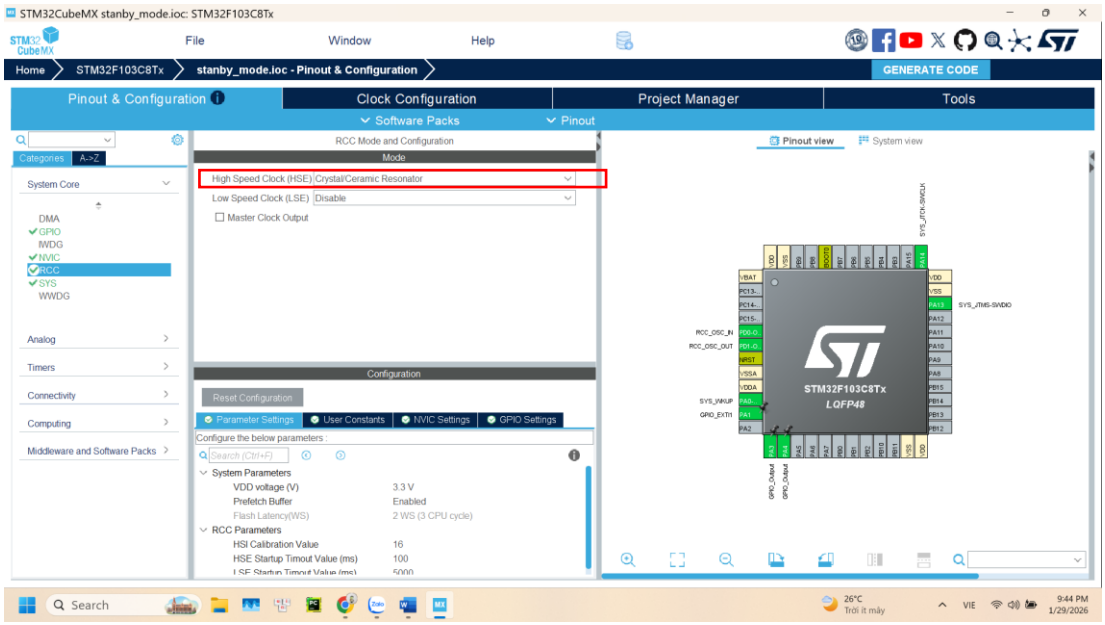


Hình 8: Cấu hình NVIC

-Mạch Standby mode có sử dụng Timer1 ngắt ngoài nên ta phải enable EXIT line1 interrupt lên.

Bước 3: cấu hình RCC (Reset and Clock Control)

-RCC có vai trò là khối điều khiển reset và xung nhịp hệ thống.

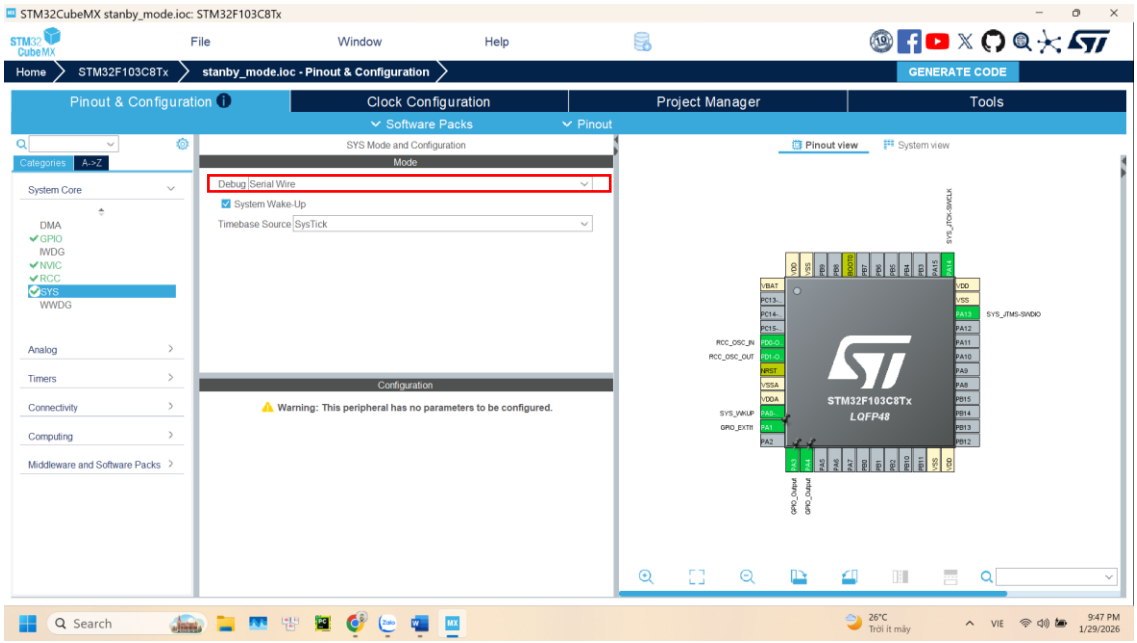


Hình 9: Cấu hình RCC

-RCC là hạ tầng clock toàn chip, nếu không cấu hình đúng toàn hệ thống không hoạt động.

-Mạch sử dụng crystal để cấp clock cho hệ thống.

Bước 4: Cấu hình SYS (System)

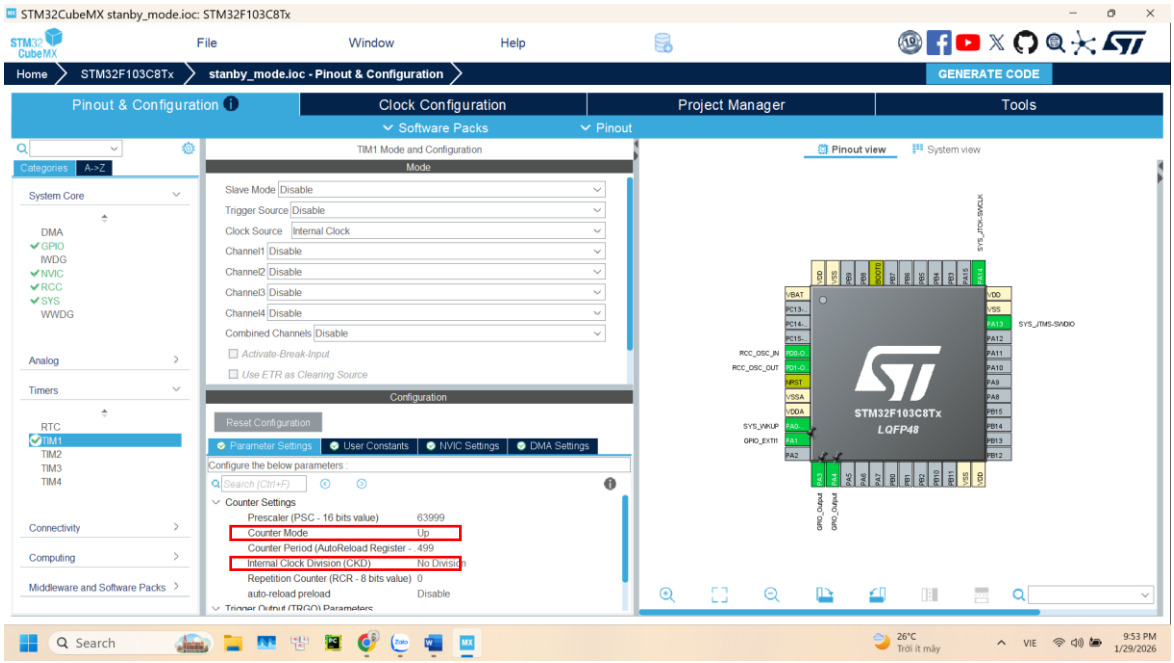


Hình 10: Cấu hình SYS

-Serial Wire Debug là giao thức debug 2 dây của ARM Cortex-M.

-Nếu không cấu hình chỗ này thì ST-link sẽ không nạp được code cho mạch.

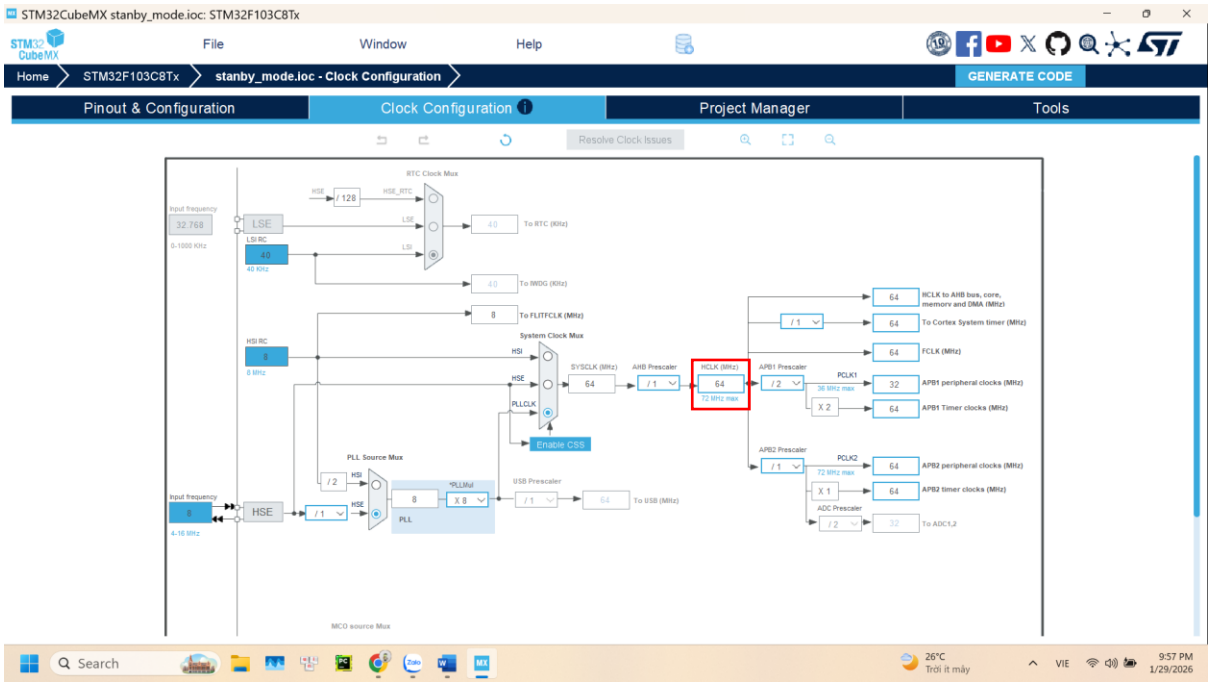
Bước 5: Cấu hình cho Timer1



Hình 11: Cấu hình Timer1

-Timer1 tạo ngắt định kỳ mỗi 500 ms với 2 thông số cần cài đặt là Prescaler và Counter Period như trong hình trên.

Bước 6: Cấu hình tần số cho toàn mạch



Hình 12: Cấu hình Clock

-Mạch hoạt động ở tần số clock là 64MHz.

3.4. Phân tích chương trình.

Code chính:

```
/* USER CODE BEGIN Header */
/**

******************************************************************************
*
  * @file           : main.c
  * @brief          : Main program body

******************************************************************************
*
  * @attention
  *
  * Copyright (c) 2025 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE
file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *

******************************************************************************
*
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------
-*/
#include "main.h"

/* Private includes ----------------------------------------------------------
-*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------
-*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------
-*/
/* USER CODE BEGIN PD */
uint8_t EnterStbFlag = 0;        // Flag: request to enter stop

uint32_t counter = 0;
/* USER CODE END PD */

/* Private macro -------------------------------------------------------------
-*/
/* USER CODE BEGIN PM */
```

```c
/* USER CODE END PM */

/* Private variables ---------------------------------------------------------
-*/
TIM_HandleTypeDef htim1;

/* USER CODE BEGIN PV */

/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------
-*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_TIM1_Init(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------
-*/
/* USER CODE BEGIN 0 */
void blink_led(){
        counter++;
        if(counter >= 1000000)
        {
                HAL_GPIO_TogglePin(GPIOA,GPIO_PIN_4);
                counter = 0;
        }
}
// Timer interrupt for led blink
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
        if(htim->Instance == htim1.Instance){
                HAL_GPIO_TogglePin(GPIOA,GPIO_PIN_3);
        }
}
/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{

  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------
-*/

  /* Reset of all peripherals, Initializes the Flash interface and the
Systick. */
  HAL_Init();
```

```c
  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_TIM1_Init();
  /* USER CODE BEGIN 2 */

        HAL_TIM_Base_Start_IT(&htim1);


  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
        if(EnterStbFlag){

                EnterStbFlag = 0;

                __HAL_PWR_CLEAR_FLAG(PWR_FLAG_WU);

                HAL_PWR_EnableWakeUpPin(PWR_WAKEUP_PIN1); // pa0 syswkup

                HAL_PWR_EnterSTANDBYMode();
        }

        if (__HAL_PWR_GET_FLAG(PWR_FLAG_SB) == 1)
        {
                __HAL_PWR_CLEAR_FLAG(PWR_FLAG_SB);
        }

        blink_led();


    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
```

```
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
  RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL8;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
  {
    Error_Handler();
  }
}

/**
  * @brief TIM1 Initialization Function
  * @param None
  * @retval None
  */
static void MX_TIM1_Init(void)
{

  /* USER CODE BEGIN TIM1_Init 0 */

  /* USER CODE END TIM1_Init 0 */

  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};

  /* USER CODE BEGIN TIM1_Init 1 */

  /* USER CODE END TIM1_Init 1 */
  htim1.Instance = TIM1;
  htim1.Init.Prescaler = 63999;
  htim1.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim1.Init.Period = 499;
  htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim1.Init.RepetitionCounter = 0;
```

```
  htim1.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if (HAL_TIM_Base_Init(&htim1) != HAL_OK)
  {
    Error_Handler();
  }
  sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
  if (HAL_TIM_ConfigClockSource(&htim1, &sClockSourceConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim1, &sMasterConfig) !=
HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM1_Init 2 */

  /* USER CODE END TIM1_Init 2 */

}

/**
  * @brief GPIO Initialization Function
  * @param None
  * @retval None
  */
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  /* USER CODE BEGIN MX_GPIO_Init_1 */

  /* USER CODE END MX_GPIO_Init_1 */

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOD_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3|GPIO_PIN_4, GPIO_PIN_RESET);

  /*Configure GPIO pin : PA1 */
  GPIO_InitStruct.Pin = GPIO_PIN_1;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /*Configure GPIO pins : PA3 PA4 */
  GPIO_InitStruct.Pin = GPIO_PIN_3|GPIO_PIN_4;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /* EXTI interrupt init*/
  HAL_NVIC_SetPriority(EXTI1_IRQn, 0, 0);
```

```
  HAL_NVIC_EnableIRQ(EXTI1_IRQn);

  /* USER CODE BEGIN MX_GPIO_Init_2 */

  /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
        if(GPIO_Pin == GPIO_PIN_1) // Stop control button
        {
                EnterStbFlag = 1;
        }

}
/* USER CODE END 4 */

/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state
*/
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line
number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line)
*/
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

-Chương trình thực hiện các chức năng chính để quan sát được Standby mode:

-Nháy LED PA4 bằng delay mềm trong vòng lặp while(1)

-Nháy LED PA3 bằng ngắt Timer TIM1

-Khi nhấn nút tại PA1, vi điều khiển:

- Thoát vòng lặp chính
- Vào chế độ Standby (low power)

-Có thể đánh thức MCU bằng chân WKUP (PA0)

-Phân tích code:

-Khai báo các biến toàn cục.

```
uint8_t EnterStbFlag = 0;        // Flag: request to enter stop
uint32_t counter = 0;
```

-EnterStbFlag:

- Cờ báo yêu cầu vào chế độ Standby
- Được set trong ngắt ngoài EXTI (PA1)
- Được kiểm tra trong while(1)

-counter:

- Biến đếm phần mềm
- Dùng để tạo trễ nháy LED PA4

-Hàm nháy LED bằng counter.

```
void blink_led(){
        counter++;
        if(counter >= 1000000)
        {
                HAL_GPIO_TogglePin(GPIOA,GPIO_PIN_4);
                counter = 0;
        }
}
```

- Hàm được gọi liên tục trong while(1)
- Mỗi lần lặp tăng counter
- Khi đạt ngưỡng → đảo trạng thái LED PA4

-Hàm ngắt Timer TIM1.

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
        if(htim->Instance == htim1.Instance){
                HAL_GPIO_TogglePin(GPIOA,GPIO_PIN_3);
        }
}
```

- Đảo trạng thái Led PA3 mỗi khi ngắt TIM1 tràn.

-Cấu hình timer:

```
  htim1.Instance = TIM1;
  htim1.Init.Prescaler = 63999;
```

```
  htim1.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim1.Init.Period = 499;
```

-Chia tần cho Timer với tần số 64MHz. Timer đếm 1000 xung/giây.

-TIM1 được cấu hình với prescaler 63999 và period 499 để tạo ngắt chu kỳ 0.5 giây. Mỗi khi Timer tràn, ngắt xảy ra và hàm callback HAL_TIM_PeriodElapsedCallback() được gọi để đảo trạng thái LED PA3.

-Ngắt ngoài EXTI – nút nhấn PA1.

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
        if(GPIO_Pin == GPIO_PIN_1) // Stop control button
        {
                EnterStbFlag = 1;
        }

}
```

-Khi nhấn nút → có tác động cạnh lên tại PA1.

-Set cờ EnterStbFlag.

-Luồng chính trong while(1)

```
while (1)
  {
        if(EnterStbFlag){

                EnterStbFlag = 0;

                __HAL_PWR_CLEAR_FLAG(PWR_FLAG_WU);

                HAL_PWR_EnableWakeUpPin(PWR_WAKEUP_PIN1); // pa0 syswkup

                HAL_PWR_EnterSTANDBYMode();
        }

        if (__HAL_PWR_GET_FLAG(PWR_FLAG_SB) == 1)
        {
                __HAL_PWR_CLEAR_FLAG(PWR_FLAG_SB);
        }

        blink_led();


    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}
```

1. Kiểm tra yêu cầu Standby mode.

- Nếu có cờ *EnterStbFlag* thì chuẩn bị vào Standby mode.

2. Chuẩn bị Standby mode.

- Xóa cờ wakeup cũ.
- Bật chân PA0 (WKUP).
- Gọi *HAL_PWR_EnterSTANDBYMode()*.

-Sau lệnh này:

- MCU reset hoàn toàn.
- RAM mất dữ liệu.
- Chương trình chạy lại từ main().

3. Kiểm tra cờ Standby mode.

```
if (__HAL_PWR_GET_FLAG(PWR_FLAG_SB) == 1)
```

- Kiểm tra xem MCU có vừa mới thoát Standby hay không.
- Nếu =1 thì là vừa wake-up từ Standby mode.

```
__HAL_PWR_CLEAR_FLAG(PWR_FLAG_SB);
```

- Thực hiện xóa cờ Standby.

4. Gọi hàm blink_led và bắt đầu lại vòng lặp

-Cấu hình xung hệ thống

```
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
  RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL8;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
```

```
  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
  {
    Error_Handler();
  }
}
```

-Khối cấu hình xung nhịp thiết lập nguồn clock cho toàn bộ vi điều khiển. Chương trình sử dụng thạch anh ngoài HSE làm nguồn clock chính, sau đó đưa qua PLL để nhân tần số. Với HSE = 8 MHz và hệ số nhân PLL = 8, xung hệ thống đạt 64 MHz. Đây là mức xung phù hợp cho STM32F103, đảm bảo vi điều khiển hoạt động ổn định và đủ tốc độ xử lý cho ngắt và Timer.

-Cấu hình GPIO

```
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  /* USER CODE BEGIN MX_GPIO_Init_1 */

  /* USER CODE END MX_GPIO_Init_1 */

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOD_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3|GPIO_PIN_4, GPIO_PIN_RESET);

  /*Configure GPIO pin : PA1 */
  GPIO_InitStruct.Pin = GPIO_PIN_1;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /*Configure GPIO pins : PA3 PA4 */
  GPIO_InitStruct.Pin = GPIO_PIN_3|GPIO_PIN_4;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /* EXTI interrupt init*/
  HAL_NVIC_SetPriority(EXTI1_IRQn, 0, 0);
  HAL_NVIC_EnableIRQ(EXTI1_IRQn);

  /* USER CODE BEGIN MX_GPIO_Init_2 */

  /* USER CODE END MX_GPIO_Init_2 */
}
```

-Cấu hình các chân PA3 và PA4 ở chế độ Output push-pull để điều khiển LED.

-Cấu hình chân PA1 và PA0 ở chế độ Input kết hợp ngắt ngoài (EXTI) để nhận tín hiệu từ nút nhấn.

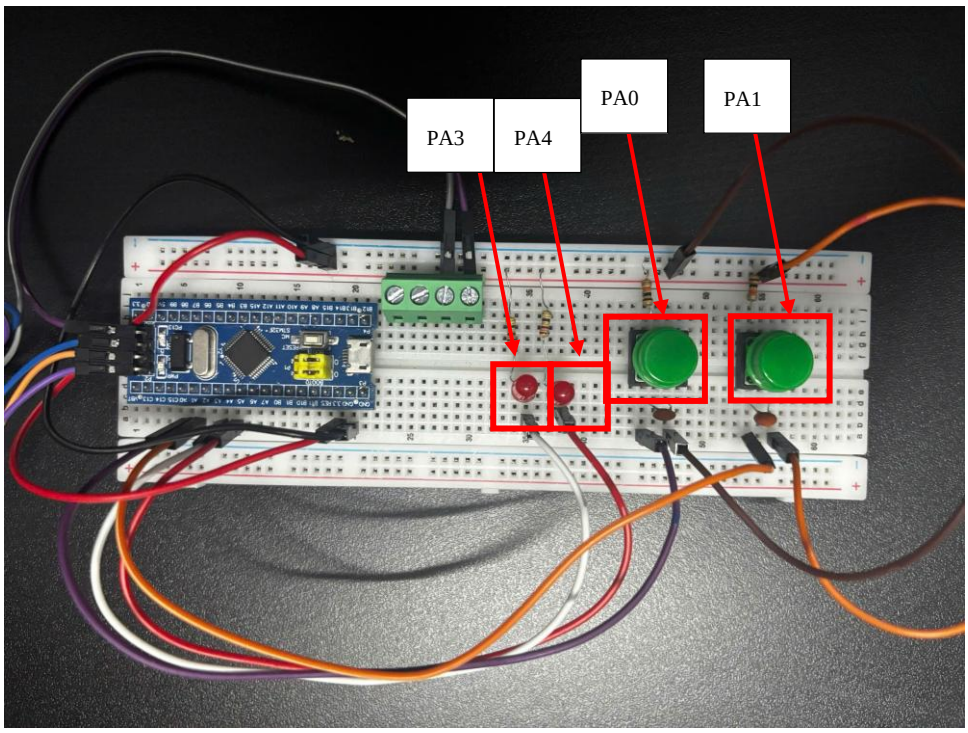
-Xử lý lỗi hệ thống

```
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state
*/
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}
```

-Hàm xử lý lỗi được gọi khi xảy ra lỗi trong quá trình cấu hình clock, Timer hoặc GPIO. Khi lỗi xảy ra, chương trình tắt toàn bộ ngắt và giữ vi điều khiển trong vòng lặp vô hạn.

CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ PHÂN TÍCH.

4.1. Kết quả phần cứng



Hình 13: Kết quả phần cứng

-Nút nhấn PA1: Vào chế độ Standby.

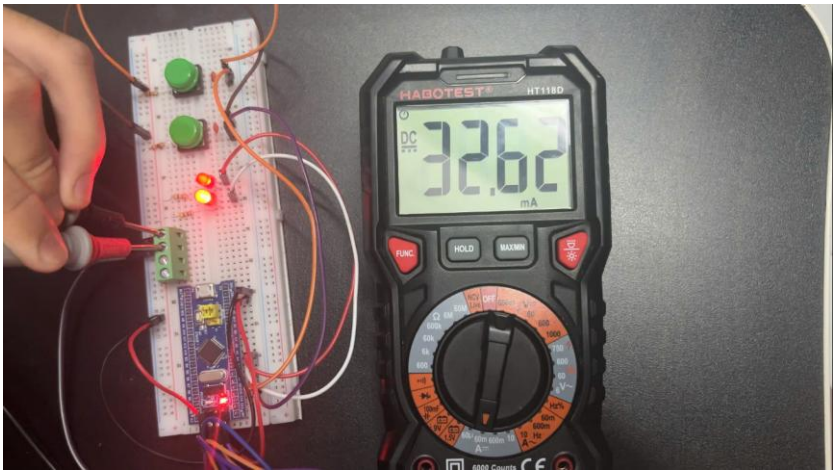
-Nút nhấn PA0: Thoát chế độ Standby.

-Led PA3: Nhấp nháy theo hàm Blink_led().

-LED PA4: Đảo trạng thái khi ngắt TIM1 tràn.

4.2. Kết quả đo lường tiêu thụ dòng điện của chế độ Standby.

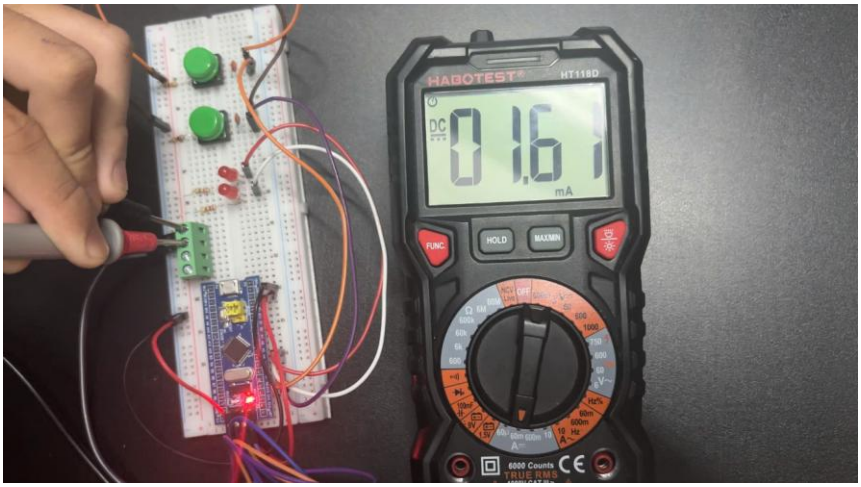
-Trước khi vào chế độ Standby



Hình 14: dòng tiêu thụ Run mode

- Khi ở chế độ Run, mạch hoạt động với dòng tiêu thụ ~32.62 mA

-Sau khi vào chế độ Standby.



Hình 15: Dòng tiêu thụ Standby mode

- Khi ở chế độ Standby mạch hoạt động với dòng tiêu thụ ~1.67 mA.

4.3. Phân tích kết quả đo lường dòng điện tiêu thụ.

-Theo datasheet STM32F103C8T6 ở chế độ Standby có dòng tiêu thụ chỉ ở mức khoảng 2µA, nhưng khi đo trên bo mạch thực tế Blue Pill với STLink rời, dòng tiêu thụ đạt khoảng 1.67 mA. Nguyên nhân chính là do LED nguồn trên bo mạch.

-LED nguồn được nối trực tiếp với đường 3.3 V thông qua điện trở hạn dòng và luôn sáng theo trạng thái hoạt động của STM32. Với điện áp rơi thuận của LED khoảng 2 V, dòng tiêu thụ ước tính vào khoảng 1–1.3 mA, gần tương đương giá trị dòng đo được.

-Do đó, dòng tiêu thụ lớn ở chế độ Standby không phản ánh mức tiêu thụ của vi điều khiển, mà chủ yếu đến từ LED nguồn của bo mạch phát triển.

CHƯƠNG 5: KẾT LUẬN VÀ ĐÁNH GIÁ.

5.1. Kết luận giá trị dòng điện tiêu thụ.

-Giá trị dòng điện tiêu thụ đo được ở chế độ Standby trên bo mạch STM32F103C8T6 Blue Pill vào khoảng 1.67 mA là phù hợp với thực tế sử dụng bo mạch phát triển. Giá trị này không phản ánh dòng tiêu thụ của riêng vi điều khiển, mà chủ yếu do LED nguồn luôn hoạt động gây ra.

-Trong điều kiện mạch tối giản, không có LED nguồn và linh kiện phụ trợ, dòng tiêu thụ của STM32F103C8T6 ở chế độ Standby có thể đạt mức vài µA, đúng như thông số công bố trong datasheet của nhà sản xuất.

5.2. Đánh giá hiệu quả của chế độ Standby.

- Tối ưu về mặt năng lượng: Chế độ Standby được đánh giá là trạng thái tiết kiệm năng lượng nhất của STM32F1 nhờ việc ngắt hoàn toàn bộ điều áp và miền cấp nguồn 1.8V.

- Có thể gây mất dữ liệu nếu như chưa được backup vào bộ nhớ.

5.3. Ứng dụng thực tế của chế độ Standby.

-Thiết bị chạy pin dài hạn: Standby là lựa chọn hàng đầu cho các ứng dụng chạy pin không yêu cầu hoạt động liên tục và cần kéo dài tuổi thọ nguồn điện tối đa như thiết bị cầm tay (handheld equipment) hoặc các bộ điều khiển di động.

-Hệ thống cảm biến công nghiệp: Trong môi trường công nghiệp, chế độ này được ứng dụng cho các bộ cảm biến chờ đợi sự kiện bên ngoài để "thức dậy" và thực hiện các tác vụ xử lý hoặc gửi dữ liệu, sau đó quay lại trạng thái nghỉ sâu.

TÀI LIỆU THAM KHẢO

[1] STMicroelectronics, *AN2629: STM32F101xx, STM32F102xx and STM32F103xx low-power modes*, Doc. ID 13922, Rev. 2, Apr. 2009.

[2] STMicroelectronics, *STM32F103x8, STM32F103xB Datasheet – Medium-density performance line Arm®-based 32-bit MCU*, DS5319, Rev. 20, Jul. 2025.

[3] STMicroelectronics, *RM0008: STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx Advanced ARM-based 32-bit MCUs – Reference manual*, Doc. ID 13902, Rev. 9, Jun. 2009.

[4] Next Electronics, *ARM Cortex-M Series Architecture Tutorial*, online tutorial. Available:***https://www.next.gr/tutorials/microcontrollers-and-development-boards/arm-cortex-m-series-architecture-tutorialz***